IMY 220

Assignment 3

React, Express and MongoDB

Due: Wednesday, 09 September 2026 at 13:00

The submission instructions are available on ClickUP. Any deviation from these instructions will result in a 10% deduction from your mark.

Instructions

This is an individual, take-home assignment.

This assignment focuses on:

- connecting an Express server to MongoDB;
- retrieving data from MongoDB;
- inserting data into MongoDB;
- creating API endpoints with Express;
- fetching data from an API using React;
- using `useEffect()` to load data;
- sending data from React to an Express server;
- updating React state after receiving data from an API;
- controlled forms and user input;
- basic validation and error handling.

A full-stack web application consists of multiple layers that communicate with one another.

In this assignment:

- React is responsible for the user interface;
- Express is responsible for the API;
- MongoDB is responsible for persistent data storage.

The React application must communicate with MongoDB through the Express server.

React must not connect directly to MongoDB.

Scenario

You are continuing your work on the PhotoShare social media platform.

In Assignment 2, posts were stored temporarily in React state. This meant that any posts added by the user disappeared when the page was refreshed.

PhotoShare now requires posts to be stored permanently.

Another developer has created the basic React, Express and MongoDB project structure.

Your task is to complete the application so that PhotoShare posts can be:

- retrieved from MongoDB;
- displayed in the React interface;
- created using a React form;
- sent to the Express API;
- stored in MongoDB;
- displayed immediately after being successfully added.

Supplied Files

Download the supplied project from ClickUP.

The project contains two main folders:

```
assignment-3/
│
├── client/
│
└── server/
```

The `client` folder contains the React application.

The `server` folder contains the Express application and MongoDB connection.

Do not substantially change the supplied project structure unless instructed.

Replace uXXXXXXX with your student number where indicated.

Ensure that your student number is included as a comment in all JavaScript and JSX files that you modify.

Client

The supplied React application contains the basic project structure required to display and add PhotoShare posts.

A stylesheet has been supplied for the application. You may modify or extend the supplied styling if required, but styling is not the primary focus of this assignment.

The application must be run using the supplied Vite development environment.

Install the required dependencies using:

npm install

Run the React application using:

npm run dev

Server

The supplied Express application contains:

- the basic Express server;
- middleware configuration;
- the MongoDB connection file;
- incomplete API functionality.

Install the required dependencies using:

npm install

Run the Express server using the supplied npm script.

Do not change the supplied server port unless instructed.

A .env.example file has been supplied. Create a .env file in the server folder and add your MongoDB connection string using the variable name shown in .env.example.

Do not rename the supplied environment variable.

MongoDB

A MongoDB database must be used to store the PhotoShare posts.

The database must contain a collection called:

posts

Each post must follow the following structure:

```
{
   username: "@sarah",
   caption: "Studying React today!"
}
```

MongoDB will automatically create an `_id` value for each stored document.

Do not manually create your own MongoDB `_id`.

Required Application Flow

Your completed application must follow this structure:

```
React
  ↓
Express API
  ↓
MongoDB
```

When retrieving posts:

```
MongoDB
  ↓
Express API
  ↓
React
```

React must never access the MongoDB database directly.

Section 1: Connect Express to MongoDB

Complete the supplied MongoDB connection functionality.

The Express server must successfully connect to the required MongoDB database.

The connection functionality must make the database available to the API routes.

The application should display an appropriate console message when the database connection has been established successfully.

Do not repeatedly create a new MongoDB connection for every API request.

Section 2: Retrieve Posts from MongoDB

Create the following Express API endpoint:

GET /api/posts

When this endpoint receives a request:

1. Access the `posts` collection.
2. Retrieve all posts from MongoDB.
3. Convert the MongoDB result into an array.
4. Return the posts to the client as JSON.

The endpoint must use MongoDB functionality such as:

find().toArray()

If no posts exist, the endpoint should return an empty array.

If an error occurs while retrieving the posts, return an appropriate error response.

Section 3: Add a Post to MongoDB

Create the following Express API endpoint:

POST /api/posts

The endpoint must receive:

- `username`;
- `caption`.

The values must be received from the request body.

Validate that both values have been supplied.

If either the username or caption is missing or empty:

- do not insert the post into MongoDB;
- return an appropriate error response.

If the submitted data is valid:

1. Create a new post object containing the username and caption.
2. Insert the post into the `posts` collection.
3. Retrieve the `_id` generated by MongoDB.
4. Return the newly created post to the React application as JSON.

The returned object should contain:

```
{
   _id: generatedMongoDBId,
   username: enteredUsername,
   caption: enteredCaption
}
```

Use an appropriate HTTP status code when a new post has been created successfully.

Section 4: Load Posts in React

Store the posts in state in the main React application.

When the React application first loads, request the posts from:

GET /api/posts

Use `useEffect()` so that the request is performed when the component first loads.

Use `fetch()` to communicate with the Express API.

When the posts have been received successfully:

1. convert the response to JSON;
2. store the returned posts in React state;
3. display the posts in the interface.

The application must not repeatedly request the posts unnecessarily.

Section 5: Display the Posts

Create reusable React components to display the posts.

Your application must include:

```
App
├── AddPostForm
└── PostList
    └── PostCard
```

You may create additional components if required.

`PostList` must receive the posts through props.

Use `map()` to generate one reusable `PostCard` for every post.

Each `PostCard` must display:

- the username;
- the caption.

Use the MongoDB `_id` as the React key when rendering posts.

For example:

key={post._id}

Do not use the array index as the key.

When there are no posts to display, show an appropriate message such as:

No posts yet.

Section 6: Add a Post from React

Create a component called:

AddPostForm

The form must contain the following inputs:

- Username;
- Caption.

Both inputs must be controlled using React state.

When the form is submitted:

1. Prevent the page from refreshing.
2. Validate that both fields contain a value.
3. If either value is empty, do not submit the post.
4. Send a POST request to: /api/posts
5. Send the username and caption as JSON in the request body.
6. Include the appropriate request headers.
7. Wait for the Express server to return the newly created post.
8. Add the returned post to the posts stored in React state.
9. Clear both form fields.

The page must not be refreshed to display the newly added post.

Do not create a temporary `_id` in React.

The application must use the post returned by the Express server because this object contains the `_id` generated by MongoDB.

Section 7: Loading and Error Handling

The application must provide basic feedback to the user.

While the initial posts are being requested from the server, display an appropriate message such as:

Loading posts...

If the posts cannot be retrieved, display an appropriate error message.

If a post cannot be added, display an appropriate error message.

Errors should be displayed in the application interface and not only inside the browser console.

The application must remain usable after an error occurs.

Section 8: Application Structure and Styling

Your application must:

- use functional React components;
- use React state for values that change;
- use `useEffect()` to retrieve the initial posts;
- use controlled inputs for the add-post form;
- use props to pass data and functionality between components;
- use `map()` to display posts;
- use MongoDB `_id` values as React keys;
- retrieve posts through the Express API;
- add posts through the Express API;
- store added posts in MongoDB;
- validate submitted data;
- provide basic loading and error feedback;
- avoid unnecessary page refreshes;
- remain functional after new posts are added.

A stylesheet has been supplied for the application. You may modify or extend the supplied styling if required, but styling is not the primary focus of this assignment.

The styling does not need to match a supplied example exactly, but the following parts should be visually distinguishable:

- page heading;
- add-post form;
- individual post cards;
- buttons;
- loading message;
- error messages;
- empty-post message.

Use reusable components where appropriate and avoid duplicating JSX.

Expected Application Behaviour

When the Express server starts:

- it connects successfully to MongoDB;
- the API becomes available to the React application.

When the React application first loads:

- a loading message is displayed while the posts are retrieved;
- React sends a request to the Express API;
- Express retrieves the posts from MongoDB;
- the posts are returned to React;
- the posts are displayed using reusable components.

When a user adds a post:

- the form values are validated;
- React sends the post to Express;
- Express validates the submitted data;
- Express inserts the post into MongoDB;
- MongoDB generates an `_id`;
- Express returns the newly created post;
- React adds the returned post to state;
- the new post appears immediately without refreshing the page.

If the page is refreshed, posts that were successfully added must still appear because they have been stored in MongoDB.

Testing Your Application

Before submitting, ensure that you have tested the following:

- the Express server connects to MongoDB;
- `GET /api/posts` returns the stored posts;
- the React application displays the stored posts;
- the loading message appears appropriately;
- a valid post can be submitted;
- the new post is stored in MongoDB;
- the new post appears immediately in React;
- the post remains available after refreshing the page;
- an empty username is rejected;
- an empty caption is rejected;
- appropriate error feedback is displayed when required.

Submission Files

Submit the complete assignment folder containing:

client/
server/

Do not include the following folders:

node_modules

Ensure that the required `package.json` files are included.

Do not submit your personal MongoDB username, password or other sensitive credentials.

Submission Instructions

ZIP the complete assignment folder using a tool such as 7-Zip or WinRAR.

Name the ZIP file:

a3-{student_number}.zip

Replace `{student_number}` with your student number.

Submit the ZIP file before the deadline to avoid a late submission.